

Mysterious Parody Bits

Lab 1 - COMP211 - Fall 2026

What truth lies in the center of a zero?

```
=====BEGIN TRANSMISSION=====
22537472616E67652066173652066726FD20746865207374617274E2041206361736520776974
687032064F6F652066726FD207468652068064646653E702004204120646F7367680675742E2220
7E2D424E5E6E697420426C61E630A0A5061727420313A206874740733A2F2F6269742E6C792F33
516C414872490A
=====END TRANSMISSION=====
```

In this lab you will follow a crumbly trail of nibbles and clues, starting with the encoded transmission above. Bit-by-bit you will shift your way closer to the elusive truth. In the end, you will emerge a bitwiser.

Accept the following assignment: <https://classroom50.org/COMP211-fa26/comp211-fa26/assignments/lab-1/accept>

Clone your Lab 1 repository After cloning, change your working directory to the cloned repository.

Part 0 of 3. A hex spell reversed is xeh

In Part 0, you will write a C program named `xeh.c` that reads a hex-encoded file and outputs the decoded ASCII text.

Encoding used to generate the input

You are given a message that was originally plain ASCII text. That text was converted into hexadecimal, where each hex digit is stored as an ASCII character. Your job is to reverse this process and print the original message.

About the input:

- The input will always contain an even number of hex characters.
- Hex letters may be upper- or lower-case.

Requirements:

- Do not use library conversion functions (e.g., `strtol`, `sscanf`). Implement the conversion yourself.
- Ignore newline characters in the input.
- Include the following header at the top of `xeh.c`:

```
// PID: 123456789
// I pledge the COMP211 honor code.
```

Basic Example

Input: 48656C6C6F

This hex string represents ASCII characters, where each pair of hex digits corresponds to one character:

```
48 -> 'H'
65 -> 'e'
6C -> 'l'
6C -> 'l'
6F -> 'o'
```

Output: Hello

To complete this process, your program will follow these steps:

1. Read the input one character at a time. Each character you read is an ASCII character such as `'4'` or `'A'`, not a number.
2. Convert each hex character to its numeric value. For example, the character `'4'` must be converted to the number 4, and `'A'` or `'a'` must be converted to 10. This conversion is done manually using arithmetic and bitwise operations, not library functions.
3. Process two hex characters at a time. The first hex digit represents the high 4 bits, and the second represents the low 4 bits.
4. Combine the two values into one byte. For example, combining 4 and 8 produces `0x48`, which corresponds to `'H'`.

A detailed walkthrough of these steps is provided below.

Step-by-Step Guide

This program is a good example of how to solve a larger problem by breaking it into small, manageable pieces and building the solution incrementally.

Instead of trying to write the entire program at once, you should:

1. Solve one small piece at a time (as outlined below).
2. Test each piece as you go. This makes bugs easier to find and fix.

This is the mindset you should carry forward in this course and all other courses: large problems become manageable when you decompose them into smaller problems and verify each step before moving on.

Step 0: Check out your current file tree.

```
learncli$ tree
```

You should see the following:

```
learncli$ tree
.
├── clues
│   ├── 00-nibbles.hex
│   └── 03-doughnuts.hex
├── hex.c
├── parity.c
└── tests
    ├── hex
    ├── parity
    ├── xeh
    │   ├── actual
    │   ├── expected
    │   └── input
    └── ytirap
```

Step 1: Read the input character-by-character.

- Create a new file named `xeh.c` inside the root directory of your Lab 1 repo.
- Copy the program below into `xeh.c`.
- This program is simply reading input one character at a time and printing it back out.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int temp;
    while ((c = getchar()) != EOF) {
        putchar(temp);
    }
    return EXIT_SUCCESS;
}
```

- Create the file `tests/xeh/input/hello.hex` using the command below:

```
learncli$ touch tests/xeh/input/hello.hex
```

- Add the test contents using the command below. By default, `echo` will add a newline.

```
learncli$ echo "48656C6C6F" > tests/xeh/input/hello.hex
```

- Open the file and notice that there is a newline at the end of the file.
- Compile and run `xeh` with input redirected from `tests/xeh/input/hello.hex`:

```
learncli$ gcc xeh.c -Wall -Wextra -std=c11 -g -o xeh
learncli$ ./xeh < tests/xeh/input/hello.hex
48656C6C6Flearncli$
```

Great, we now have a program that reads input character-by-character from `stdin` and outputs each character to `stdout`.

Step 2: Convert each hex character to its numeric value.

Each character read from the file is an ASCII character (e.g. `'4'` or `'C'`), but to decode the input, we must convert it into its numeric value.

For the input 48656C6C6F, the characters map to numbers like this:

Hex Char	Numeric Value
'4'	4
'8'	8
'6'	6
'5'	5
'C'	6
'C'	12
'6'	6
'C'	12
'F'	15

- Replace `xeh.c` with the program below.
- This program reads each character and prints:
 - the character itself
 - its ASCII decimal value
 - its converted hex value
- The newline character is ignored (if `(c == '\n') continue;`)

Here is an [ASCII table](#) for your reference.

```
#include <stdio.h>
#include <stdlib.h>

#define ASCII_OFFSET 10

int char_to_decimal(char c);

int main(void) {
    int c, value;
    while ((c = getchar()) != EOF) {
        if (c == '\n') continue;
        value = char_to_decimal(c);
        printf(
            "c (as char) = %c, c (as decimal) = %d, numeric value = "
            "%d\n",
            c, value);
        return EXIT_SUCCESS;
    }

    int char_to_decimal(char c) {
        if ('0' <= c && c <= '9') return c - '0';
        if ('A' <= c && c <= 'F') return c - 'A' + ASCII_OFFSET;
        if ('a' <= c && c <= 'f') return c - 'a' + ASCII_OFFSET;
        return -1;
    }
}
```

Run this program:

```
learncli$ gcc xeh.c -Wall -Wextra -std=c11 -g -o xeh
learncli$ ./xeh < tests/xeh/input/hello.hex
c (as char) = 4, c (as decimal) = 52, numeric value = 4
c (as char) = 8, c (as decimal) = 56, numeric value = 8
c (as char) = 6, c (as decimal) = 54, numeric value = 6
c (as char) = 5, c (as decimal) = 53, numeric value = 5
c (as char) = 6, c (as decimal) = 54, numeric value = 6
c (as char) = C, c (as decimal) = 67, numeric value = 12
c (as char) = 6, c (as decimal) = 54, numeric value = 6
c (as char) = C, c (as decimal) = 67, numeric value = 12
c (as char) = 6, c (as decimal) = 54, numeric value = 6
c (as char) = F, c (as decimal) = 70, numeric value = 15
```

Take a look at the output of the program and make sure you understand it before moving on.

Step 3 Process two hex digits at a time.

So far, you have: 1. Read input one hex digit at a time. 2. Converted a single hex character into its numeric value.

Now we take the next step: combine two hex digits to form one byte, which corresponds to a single ASCII character.

- Each ASCII character is stored in one byte (8 bits).
- Each hex digit represents 4 bits, which is called a nibble.
 - The first digit is the high nibble (upper 4 bits).
 - The second digit is the low nibble (lower 4 bits).

Example: Hex pairs from the example input: 48656C6C6F

High Nibble	Low Nibble	Hex Pair
4	8	0x48
6	5	0x65
6	12	0x6C
6	12	0x6C
6	15	0x6F

You'll use bitwise operations to combine the two nibbles. Looking at the same data in binary makes the process clearer:

High Nibble	Low Nibble	Binary Pair	Hex Pair
0b0100	0b1000	0b0100_1000	0x48
0b0110	0b0101	0b0110_0101	0x65
0b0110	0b1100	0b0110_1100	0x6C
0b0110	0b1100	0b0110_1100	0x6C
0b0110	0b1111	0b0110_1111	0x6F

- Copy the code below into `xeh.c`.
- Fill in each `/* Your code here */`.
- Be sure you are using bitwise operations to combine the two nibbles.
- Your implementation cannot use magic numbers.

```
#include <stdio.h>
#include <stdlib.h>

#define ASCII_OFFSET 10

int char_to_decimal(char c);

int main(void) {
    int c;
    unsigned char combined;
    while ((c = getchar()) != EOF) {
        if (c == '\n') continue;
        /* Your code here: The first character we read corresponds to the high
         * nibble. Convert this character to decimal. */
        printf("hi nibble: %d, ", hi);
        /* Your code here: Read next character. */
        /* Your code here: The second character we read corresponds to the low
         * nibble. Convert this character to decimal. */
        printf("lo nibble: %d, ", low);
        /* Your code here: Combine the high nibble and the low nibble to create
         * the hex pair. */
        printf("combined: %d\n", combined);
        return EXIT_SUCCESS;
    }

    int char_to_decimal(char c) {
        if ('0' <= c && c <= '9') return c - '0';
        if ('A' <= c && c <= 'F') return c - 'A' + ASCII_OFFSET;
        if ('a' <= c && c <= 'f') return c - 'a' + ASCII_OFFSET;
        return -1;
    }
}
```

Make sure your output looks like the following before moving on.

```
learncli$ gcc xeh.c -Wall -Wextra -std=c11 -g -o xeh
learncli$ ./xeh < tests/xeh/input/hello.hex
hi nibble: 4, low nibble: 8, combined: 48
hi nibble: 6, low nibble: 5, combined: 65
hi nibble: 6, low nibble: 12, combined: 6c
hi nibble: 6, low nibble: 12, combined: 6c
hi nibble: 6, low nibble: 15, combined: 6f
```

Step 4: Convert the byte into an ASCII Character.

At this point, all of the hard work is done! You have already:

1. Read two hex characters.
2. Converted each to its numeric value.
3. Combined them into a single byte using bitwise operations.

That byte is already the ASCII value of the character you want to print.

Example

Hex Pair	ASCII
0x48	H
0x65	e
0x6C	l
0x6C	l
0x6F	o

Once the byte is formed (e.g. `0x48`), printing the corresponding character (e.g. `'H'`) requires no additional conversion.

Update your program 1. Remove the `printf` statements from Step 3.2. After combining the high and low nibbles, print the resulting value as a character. Don't overwrite this! It should only take one line!

Your output should look like the following:

```
learncli$ gcc xeh.c -Wall -Wextra -std=c11 -g -o xeh
learncli$ ./xeh < tests/xeh/input/hello.hex
Hellolearncli$
```

Testing your program

Up to this point, you've been testing your program by running it and visually checking the output. While this can work for very small programs, it does not scale and it is easy to miss mistakes.

In this section, you will learn how to test your program the way real programmers do: by comparing your program's output against known correct results automatically.

You will create test cases using the inputs and expected outputs provided below.

Each test will consist of two files:

- one file containing the hex input
- one file containing the expected decoded output

Example setup

Step 1 Create the input file.

You have already created the file `tests/xeh/input/hello.hex`.

Step 2 Create the expected output file.

Create a file named `hello.expected` in the `tests/xeh/expected` directory.

```
learncli$ touch tests/xeh/expected/hello.expected
```

Add the expected output to the file:

```
learncli$ echo "Hello" > tests/xeh/expected/hello.expected
```

Double check the contents of the file:

```
learncli$ cat tests/xeh/expected/hello.expected
Hello
learncli$
```

Step 3 Run your test.

```
learncli$ ./xeh < tests/xeh/input/hello.hex > tests/xeh/actual/hello.actual
```

The above command does three things:

1. `./xeh`: Run your program.
2. `< tests/xeh/input/hello.hex`: Redirects the contents of `tests/xeh/input/hello.hex` into your program as input.
3. `> tests/xeh/actual/hello.actual`: Redirects your program's output into a file named `hello.actual` instead of printing it to the terminal.

Then run this command to compare your program's output to the expected output.

```
learncli$ diff tests/xeh/actual/hello.actual tests/xeh/expected/hello.expected
```

The `diff` command compares two files line by line:

- `tests/xeh/actual/hello.actual`: what your program produced
- `tests/xeh/expected/hello.expected`: the correct (expected) output
- No output from `diff` means that the files are identical (the test passed).
- Any output from `diff` means the files differ (the test failed).
 - `diff` will show you where the differences are.

You can see for yourself that the contents of the files are the same.

```
learncli$ cat tests/xeh/actual/hello.actual
Hellolearncli$
learncli$ cat tests/xeh/expected/hello.expected
Hellolearncli$
```

If you were wondering how the second `cat` is on a new line, it's because I pressed `Enter`. So this is what I typed in the terminal:

1. `cat tests/xeh/actual/hello.actual`
2. `<Enter>`: to get to make the command loop nearer on a new line.
3. `cat tests/xeh/expected/hello.expected`

Example 2

Step 1 Create the input file.

Create a file named `digits.hex` in `tests/xeh/input`.

```
learncli$ touch tests/xeh/input/digits.hex
```

Add the input to the file:

```
learncli$ echo "30313233343536373839" > tests/xeh/input/digits.hex
```

Double check the contents of the file:

```
learncli$ cat tests/xeh/input/digits.hex
30313233343536373839
learncli$
```

Step 2 Create the expected output file.

Create a file named `digits.expected` in the `tests/xeh/expected` directory.

```
learncli$ touch tests/xeh/expected/digits.expected
```

Add the expected output to the file:

```
learncli$ echo "0123456789" > tests/xeh/expected/digits.expected
```

Double check the contents of the file:

```
learncli$ cat tests/xeh/expected/digits.expected
0123456789
learncli$
```

Again, notice that the shell prompt is on the same line as the output. That is because there is no new line at the end of `tests/xeh/expected/digits.expected`. This is expected.

Step 3 Run your test.

```
learncli$ ./xeh < tests/xeh/input/digits.hex > tests/xeh/actual/digits.actual
```

Then run this command to compare your program's output to the expected output.

```
learncli$ diff tests/xeh/actual/digits.actual tests/xeh/expected/digits.expected
```

The `diff` command compares two files line by line. No output from `diff` means that the files are identical (the test passed).

You can see for yourself that the contents of the files are the same:

```
learncli$ cat tests/xeh/actual/digits.actual
0123456789learncli$
learncli$ cat tests/xeh/expected/digits.expected
0123456789learncli$
```

Pause for Questions

Before continuing, answer the [Gradescope questions on diff](#).

Example 3.

Test 3: All Uppercase Letters

We'll walk through one more example together.

Input file: `tests/xeh/input/upper.hex`

```
414243444546
```

Expected output file: `tests/xeh/expected/upper.expected`

```
ABCDE
F
```

Step 1 In the previous example, we created the file (with `touch`) and wrote to the file (with `echo`) in two separate steps. You can actually do this in one step because `echo` will create the file if it does not exist.

```
learncli$ echo "414243444546" > tests/xeh/input/upper.hex
learncli$ echo "ABCDEF" > tests/xeh/expected/upper.expected
```

Step 2 Run your program.

```
learncli$ ./xeh < tests/xeh/input/upper.hex > tests/xeh/actual/upper.actual
```

Step 3 Compare your program's output to the expected output.

```
learncli$ diff tests/xeh/actual/upper.actual tests/xeh/expected/upper.expected
```

- No output from `diff` means that the files are identical (the test passed).
- Any output from `diff` means the files differ (the test failed).
 - `diff` will show you where the differences are.

Now, it's your turn to create the remaining tests.

This is great practice for your Unix command line skills!

Test 4: All lowercase Letters

Input file: `tests/xeh/input/lower.hex`

```
616263646566
```

Expected output file: `tests/xeh/expected/lower.expected`

```
abcde
f
```

Once you've created the test files, run the test:

```
learncli$ ./xeh < tests/xeh/input/lower.hex > tests/xeh/actual/lower.actual
learncli$ diff tests/xeh/actual/lower.actual tests/xeh/expected/lower.expected
```

Test 5: Punctuation and Symbols

Input file: `tests/xeh/input/punct.hex`

```
21223F2E
```

Expected output file: `tests/xeh/expected/punct.expected`

```
!?,
.
```

This one is tricky because `!`, `*`, and `?` all have special meanings in the shell, so you need to quote or escape them properly before redirecting to a file. There are two ways you can write these contents to `punct.out`:

Option 1: Single quotes are simplest because they prevent all shell expansion:

```
learncli$ echo '!*,?' > tests/xeh/expected/punct.expected
```

Option 2: You can also escape the special characters explicitly:

```
learncli$ echo '\!*\?.' > tests/xeh/expected/punct.expected
```

Remember, you can verify the output:

```
learncli$ cat tests/xeh/expected/punct.expected
```

Once you've created the test files, run the test. The commands to run the test are not provided here as this is your chance to get practice generating the commands yourself.

Test 6: Includes Spaces

Input file: `tests/xeh/input/spaces.hex`

```
48656C6C6F20576F726C64
```

Expected output file: `tests/xeh/expected/spaces.expected`

```
Hello World
```

Once you've created the test files, run the test. The commands to run the test are not provided here as this is your chance to get practice generating the commands yourself.

Test 7: Includes Newlines

Input file: `tests/xeh/input/newLines.hex`

```
4C696E6520310A0A4C696E6520320A
```

Expected output file: `tests/xeh/expected/newLines.expected`

```
Line 1
Line 2
```

This one also requires special handling:

Option 1: use `printf`:

```
learncli$ printf "Line 1\nLine 2\n" > tests/xeh/expected/newLines.expected
```

Option 2: Some shells support `\E` for escape sequences:

```
learncli$ echo -E "Line 1\nLine 2\n" > tests/xeh/expected/newLines.expected
```

Remember to check your output:

```
learncli$ cat tests/xeh/expected/newLines.expected
```

Once you've created the test files, run the test. The commands to run the test are not provided here as this is your chance to get practice generating the commands yourself.

Deleting the temporary files

If you run `tree` right now, you will see several temporary files. They all end in `_actual`.

```
learncli$ tree tests/xeh
tests/xeh
├── actual
│   ├── digits.actual
│   ├── hello.actual
│   ├── lower.actual
│   ├── newLines.actual
│   ├── punct.actual
│   ├── spaces.actual
│   └── upper.actual
├── expected
│   ├── digits.expected
│   ├── hello.expected
│   ├── lower.expected
│   ├── newLines.expected
│   ├── punct.expected
│   ├── spaces
```